NukMiris v0.8.1 — Independent Greenfield Architecture Challenge

Deliverable 3 of 5 — Architecture Decision Records

*Independent architect submission · 26 October 2026 · Status of every record: **Proposed**, pending adversarial cross-review. Requirement IDs, F-n, T-n and S-n refer to Deliverable 1; sections (§) refer to Deliverable 2.*

Each record gives the context, the decision, the alternatives I weighed, benefits, drawbacks and risks, and the specific conditions under which I would reconsider it.

ADR	Decision	Confidence
001	Modular monolith, two process roles	High
002	TypeScript on Node.js	Medium
003	PostgreSQL as the only stateful dependency	High
004	Postgres leases, fencing and transactional inbox/outbox instead of a broker or workflow engine	Medium
005	Typed conversation state with a pure reducer and server-side opaque offers	High
006	Google Calendar as truth; client-assigned event IDs, ownership properties, ETag preconditions	Medium (depends on S-1, S-2)
007	Availability from free/busy windows plus a pure engine; uncached final recheck	High
008	Reservation arbitration by exclusion constraint with fencing	Medium (S-10)
009	Definite/uncertain outcome model; idempotent retry only where idempotent by construction	High
010	Outbound attempt ledger, opaque callback correlation, monotonic receipt merge	Medium (S-6)

ADR	Decision	Confidence
011	Reminders as durable occurrences with a polling scheduler and preflight reads	High
012	Tenant isolation by shared schema, composite keys, row-level security and role separation	Medium
013	Envelope-encrypted per-business credentials bound to tenant	High
014	Calendar change detection by polling plus preflight, not push	Medium
015	Operator access through a private admin API and CLI; no dashboard	High
016	Managed containers and managed Postgres in one region	Medium
017	OpenTelemetry, redacted logs, invariant sentinels and a canary business	High
018	Disaster recovery by PITR, write-once ingress archive and quarantine restore	Medium
019	Testing by simulators, crash-point injection and requirement-traced acceptance	High
020	Time handling by instants, IANA zones and a conservative DST policy	High
021	No LLM in the control path	High

ADR-001 — Modular monolith, two process roles

Context. A small team must deliver strong correctness and recoverability with minimal operational surface (priority 1–4, small team). **Decision.** One codebase, one database, two deployables: `web` (ingress, admin API) and `worker` (all loops). Boundaries are enforced by layering rules in CI. **Alternatives.** Microservices per capability; serverless functions per webhook and job; a single process doing everything. **Benefits.** One transactional boundary for inbox, state, ledger and outbox. One deploy. Cheap local reproduction of failures. Splitting `web` from `worker` isolates internet exposure from credentials. **Drawbacks and risks.** A monolith can erode into a tangle without enforced boundaries.

Workers and ingress scale together unless split further. **Reconsider when.** Separate teams own separate capabilities, or one loop needs very different scaling or isolation (for example Calendar sync at high tenant counts).

ADR-002 — TypeScript on Node.js

Context. Provider payloads are JSON, the official Google client is mature, and the owner has a working POC ecosystem. **Decision.** Strict TypeScript on the current LTS Node.js, Fastify, Zod at every boundary, Kysely for SQL. **Alternatives.** Go; Kotlin or Java; Python; Elixir. **Benefits.** Team continuity, one type system from webhook to database, a large hiring pool, strong provider SDKs. **Drawbacks and risks.** Single-threaded runtime (acceptable: workloads are I/O-bound). Types vanish at runtime, so Zod validation is mandatory. npm supply-chain exposure. **Mitigations.** Lockfile, provenance checks, a small dependency set, SBOM, egress allow-list. **Reconsider when.** CPU-bound work appears, or the team's skills shift decisively. This is the least technology-critical decision here; Go would also be sound.

ADR-003 — PostgreSQL as the only stateful dependency

Context. The design needs transactions across inbox, state, ledger and outbox; constraint-based arbitration; row-level security; and recoverable backups. **Decision.** Managed PostgreSQL 16+ with HA and point-in-time recovery, plus the `btree_gist` extension. **Alternatives.** SQLite (the POC); MySQL; DynamoDB or another key-value store; a document database; Postgres plus Redis. **Benefits.** One system provides ACID transactions, the exclusion constraint, RLS, `SKIP LOCKED` and PITR. There is exactly one thing to back up and restore (O03). **Drawbacks and risks.** A single primary is a scaling and availability ceiling. Extensions and RLS depend on the managed offering (S-10, S-12). **Reconsider when.** Sustained write volume approaches a single primary's limits, or an availability target exceeds what multi-AZ failover provides.

ADR-004 — Postgres leases and fencing instead of a broker or workflow engine

Context. Required semantics: per-conversation order, uncertain outcomes, never-replay reminders, and acceptance atomic with state change. **Decision.** Claimable rows with `lease_until` and a `fence`, claimed with `FOR UPDATE SKIP LOCKED`; commits are fence-checked; polling is authoritative and notifications are a latency hint. A transactional inbox and outbox live in the same database. **Alternatives.** SQS, RabbitMQ or Kafka; Redis queues; Temporal or another durable-execution engine; pg-boss or Graphile Worker. **Benefits.** No dual-write problem between a database and a broker. Domain states

(uncertain, expired, skipped) are first-class. No extra infrastructure to run or restore. **Drawbacks and risks.** I own a small piece of scheduling code that must be correct. Polling adds steady load. No built-in visibility UI. **Reconsider when.** Claim latency degrades under realistic load, workflows become long-running and multi-step enough that a workflow engine pays for itself, or a second service needs the same events.

ADR-005 — Typed conversation state, pure reducer, server-side opaque offers

Context. I03 forbids stale, guessed or foreign actions from causing effects; Section 3 defines transient views with a 15-minute life. **Decision.** State is a validated discriminated union. `decide(state, input, context) → (state', effects)` is pure. Every interactive action is an opaque 128-bit token resolving to a stored payload. **Alternatives.** Encoding state in button payloads; a framework state machine library; storing free-form session dictionaries; an LLM-driven dialogue. **Benefits.** Exhaustively testable by property tests. Stale and forged actions are impossible by construction. Labels never act as identifiers. **Drawbacks and risks.** More rows (offer sets) and a cleanup job. Rendering and validation must stay in lockstep. **Reconsider when.** Dialogue complexity grows beyond what a hand-written reducer can keep legible; a state-machine library could then be adopted behind the same pure interface.

ADR-006 — Calendar as truth; client-assigned IDs, ownership properties, ETag preconditions

Context. Section 2 makes Google the truth for appointments. Calendar and database cannot commit atomically (Section 4). **Decision.** Allocate the event ID before the write and store it on the operation. Stamp ownership and change identity in private extended properties. Use `If-Match` for changes and deletes. Keep a local projection as a cache only. **Alternatives.** Database as truth with one-way sync; searching by title or description to find events; server-assigned IDs with after-the-fact matching; a service account with a shared Calendar instead of OAuth. **Benefits.** Lost-response recovery becomes a read by ID, then an idempotent retry. Restore reconciliation can rediscover events by property. Manual edits by the owner are respected. **Drawbacks and risks.** Relies on provider behaviour I could not verify (S-1, S-2, S-3). Properties are readable by any client with Calendar access and are not a security boundary. **Reconsider when.** Any of S-1 to S-3 fails. Then fall back to read-only proof plus operator resolution and re-assess the operator workload.

ADR-007 — Free/busy windows, pure slot engine, uncached final recheck

Context. C02 requires reach across 365 civil days; C03 forbids treating failure as empty; a provider budget must be justified. **Decision.** Pure `computeSlots`; read about 31-day free/busy windows lazily; short-lived advisory cache for browsing only; fresh read of the exact interval before every write. **Alternatives.** A full local mirror of each Calendar maintained by sync; querying per day; caching availability and trusting it. **Benefits.** Correctness does not depend on cache freshness. Typical cost is one provider call per browse. Privacy-friendly (busy times only). **Drawbacks and risks.** Browse can offer a slot that disappears; the recheck produces a visible “taken” reply. Worst-case search costs about twelve calls. **Reconsider when.** Provider quota or latency becomes the constraint, which would justify a mirror kept current by sync.

ADR-008 — Reservation arbitration by exclusion constraint with fencing

Context. I06: cooperating operations must not finalize conflicting intervals; stale workers must not release successors. Multiple instances run concurrently. **Decision.** `calendar_reservations` with `EXCLUDE USING gist (calendar_id =, during &&, appointment_key <>)`, half-open ranges including buffer, fence-checked transitions. Capacity is one per Calendar (F-01). **Alternatives.** Advisory locks; application-level locks or Redis locks; relying on Google alone; a unique index over discretized slots. **Benefits.** The database arbitrates, so correctness does not depend on process behaviour. Moves may overlap their own appointment safely. **Drawbacks and risks.** Needs `btree_gist` and correct behaviour of the not-equal operator class (S-10). Capacity above one needs redesign. It cannot exclude manual writers (U-EXTERNAL). **Reconsider when.** Multi-staff capacity is required, or S-10 fails (fall back to advisory locks plus a unique slot index).

ADR-009 — Definite/uncertain outcome model; idempotent retry only where idempotent by construction

Context. I07 and I09 forbid false success and blind replay of possibly accepted writes or sends. **Decision.** Every adapter returns applied, definitely rejected, or uncertain; unknown is uncertain. Calendar creates may be retried with the *same* ID after a settle window. Sends and other non-idempotent calls are never retried automatically. **Alternatives.** Treat timeouts as failure and retry; treat timeouts as success; operator-only resolution for everything. **Benefits.** No duplicate events or messages from retries. Most uncertainty resolves automatically and provably. **Drawbacks and risks.** Customers

occasionally wait through a settle window. The settle window and the interpretation of "not blind" (F-03) need owner agreement. **Reconsider when.** Measured provider latency shows the settle window is too short or too long, or S-1 fails.

ADR-010 — Outbound ledger, callback correlation, monotonic receipt merge

Context. WhatsApp acceptance and delivery are different facts; receipts arrive late and out of order; there is no send-side idempotency. **Decision.** Commit an attempt row before each send. Send an opaque callback value equal to the attempt ID. Store receipts append-only and compute state as the maximum rank, with failure unable to lower delivered or read. **Alternatives.** Fire and forget with logs; correlate only by provider message ID; retry on timeout. **Benefits.** Lost responses can be resolved by later receipts. Reordering cannot corrupt state. A receipt cannot resolve another customer's send. **Drawbacks and risks.** Depends on the callback echo (S-6). Some sends stay uncertain until reviewed. **Reconsider when.** S-6 fails, or the provider adds a true idempotency key, which would allow safe retry.

ADR-011 — Reminders as durable occurrences, polling scheduler, preflight reads

Context. R02 and R03 require instant-based offsets, a 15-minute late allowance and no replay after downtime. **Decision.** Occurrences with absolute `due_at` and `expires_at`, one per appointment, schedule and kind; a 5-second polling scheduler; a fresh Calendar read and permission and template checks before every send. **Alternatives.** Cron jobs; delayed queue messages; cloud schedulers; in-memory timers. **Benefits.** State survives restarts. Eligibility is data, so dry-run, off and live differ only in policy. Moves and cancellations invalidate by key. **Drawbacks and risks.** Polling cadence bounds precision (fine for a 15-minute allowance). Preflight adds one provider call per send. **Reconsider when.** Volume makes per-send reads too costly, or sub-minute precision is required.

ADR-012 — Shared schema, composite keys, row-level security, role separation

Context. I01 demands strict isolation; the platform will serve many small tenants. **Decision.** One schema; `business_id` on every tenant table; composite foreign keys; forced RLS keyed on a per-transaction setting; separate roles for ingress, runtime, read-only, break-glass and migration. **Alternatives.** Schema per tenant; database per tenant; application-layer filtering only. **Benefits.** The database refuses cross-tenant access even when application code is wrong. Operationally cheap at hundreds of tenants. **Drawbacks**

and risks. RLS complexity, planner effects and pooling pitfalls (S-10). A single database is a shared blast radius. **Reconsider when.** A customer demands physical isolation, or contractual residency differs by tenant.

ADR-013 — Envelope-encrypted credentials bound to tenant

Context. I11 requires protected credentials and a separate recovery plan. **Decision.** Per-row data keys wrapped by KMS, AES-256-GCM, additional authenticated data of business, provider and credential ID. Platform secrets in a secrets manager. Only `worker` decrypts. **Alternatives.** Plaintext token files (the POC); a secrets-manager entry per business; application-wide key. **Benefits.** A stolen database dump is useless without KMS access. Ciphertext cannot be moved across tenants. Key recovery is independent of database recovery. **Drawbacks and risks.** KMS availability becomes a runtime dependency (mitigate with in-memory token caching). Key rotation needs a background job. **Reconsider when.** A per-tenant secrets manager is preferred for audit reasons and the tenant count makes its cost acceptable.

ADR-014 — Polling plus preflight for Calendar change detection

Context. R08 requires detecting manual moves and deletions. **Decision.** Per-business `updatedMin` polling about every two minutes, backing off when idle, plus a read before every reminder send. **Alternatives.** Push notification channels; syncToken incremental sync; polling only at send time. **Benefits.** No public callback, no channel renewal, simple to reason about. Preflight makes correctness independent of poll lag. **Drawbacks and risks.** Provider calls grow linearly with active tenants. **Reconsider when.** Poll volume nears quota, then add push channels as a hint while keeping polling as the safety net.

ADR-015 — Private admin API and CLI; no dashboard

Context. The baseline implies no dashboard (E-class) but requires scoped status, consent recording, reconciliation and reviewed retry. **Decision.** A private admin API with a thin CLI, SSO with MFA, roles, mandatory reasons, immutable audit, masked defaults. **Alternatives.** Direct database access; a web admin UI; shared credentials. **Benefits.** Small surface, fully audited, scriptable for runbooks. **Drawbacks and risks.** Less friendly for non-engineers; any future business-owner dashboard is separate work. **Reconsider when.** More than a few operators, or support staff who are not engineers.

ADR-016 — Managed containers and managed Postgres, one region

Context. Small team; unknown commercial targets (U-OPS); workers need always-on capacity. **Decision.** Two services on a managed container platform plus a managed multi-AZ Postgres, in one region fixed after the owner's residency decision. **Alternatives.** Kubernetes; a single VM; serverless functions; multi-region active-active. **Benefits.** Little to operate; automatic failover; reproducible by infrastructure as code. **Drawbacks and risks.** Provider lock-in at the margins. A regional outage means downtime up to the restore objective. **Reconsider when.** The availability target requires multi-region, or cost at scale favours self-management.

ADR-017 — OpenTelemetry, redacted logs, sentinels, canary

Context. Q01 requires actionable observability; diagnostics must not leak customer data. **Decision.** Traces with correlation keys, structured redacted logs, business-level SLIs, hourly invariant sentinels, a production canary business. **Alternatives.** Logs only; vendor-specific agents; metrics without invariants. **Benefits.** Failures that matter to customers (stalls, uncertainty, late reminders) are alerts rather than discoveries. Vendor-neutral. **Drawbacks and risks.** Noise and cost if alerts are not tuned. The canary needs a maintained WhatsApp number and Calendar. **Reconsider when.** Alert fatigue or telemetry cost becomes material.

ADR-018 — PITR, write-once ingress archive, quarantine restore

Context. A restore cannot undo external effects (O03); restore after side effects is a stated failure scenario. **Decision.** Continuous PITR; a write-once archive of raw signed webhook bodies; a scripted restore that starts in quarantine, replays the archive, reconciles Calendar and outbound state, and marks unknown sends uncertain. **Alternatives.** Snapshots only; accept duplicates after restore; synchronous cross-region replication. **Benefits.** Small loss window; no automatic resends; Calendar remains a cross-check. **Drawbacks and risks.** The archive export is best-effort and defines part of the loss window. The procedure is only trustworthy if drilled. **Reconsider when.** RPO needs tightening (synchronous replication) or retention rules forbid the archive.

ADR-019 — Simulators, crash-point injection, requirement-traced acceptance

Context. Q01 asks for measurement of requirements, races and failures, not line or test counts. **Decision.** Provider simulators with fault injection, a crash-point harness that kills and restarts on the same database, generated tenant-leakage tests, property tests on the pure core, and CI reports by requirement ID. **Alternatives.** Unit tests with mocks; manual

QA against sandboxes only. **Benefits.** Every dangerous window in §8.4 and §11.3 is exercised deliberately and repeatedly. **Drawbacks and risks.** Significant up-front investment. Simulators can drift from reality (mitigated by contract tests and a weekly sandbox job). **Reconsider when.** Not expected; reduce scope only by deferring, never by dropping, the crash-point suite.

ADR-020 — Instants, IANA zones, conservative DST policy

Context. I10 and U-DST: elapsed offsets use instants; civil dates use the business zone. **Decision.** Store instants plus the business zone; compute civil dates in one module with Temporal or Luxon; reject nonexistent local times; exclude repeated local times from offers; refresh tzdata through the image pipeline. **Alternatives.** Store local wall-clock times; use fixed offsets; offer both occurrences of a repeated hour. **Benefits.** No silent shift across DST or restart. Behaviour is explicit and testable. **Drawbacks and risks.** Loses one bookable hour per year in zones with repeated hours. Government rule changes after booking may shift an event relative to wall-clock expectation. **Reconsider when.** The owner approves a different DST policy, or target countries are known to have none.

ADR-021 — No LLM in the control path

Context. Section 3 specifies bounded Spanish grammar; recognition alone must never mutate (W06). **Decision.** Deterministic grammar and intent rules. If an LLM is ever added, it may only propose candidate intents behind the same reducer guard. **Alternatives.** LLM intent classification; LLM-driven dialogue. **Benefits.** Deterministic, testable, auditable behaviour and no new data-processing party for customer text. **Drawbacks and risks.** Less forgiving of free-form phrasing. **Reconsider when.** Product asks for broader natural language and the owner accepts the privacy and testing costs.